AUTOMATED REVIEW RATING SYSTEM

BALANCED DATASET

Jeevan Santhosh S

1. Project Overview

An intelligent system that analyzes customer reviews and predicts corresponding star ratings. It uses natural language processing (NLP) to extract sentiment and key features from textual feedback. The model is trained on labelled review data to learn patterns between language and rating.

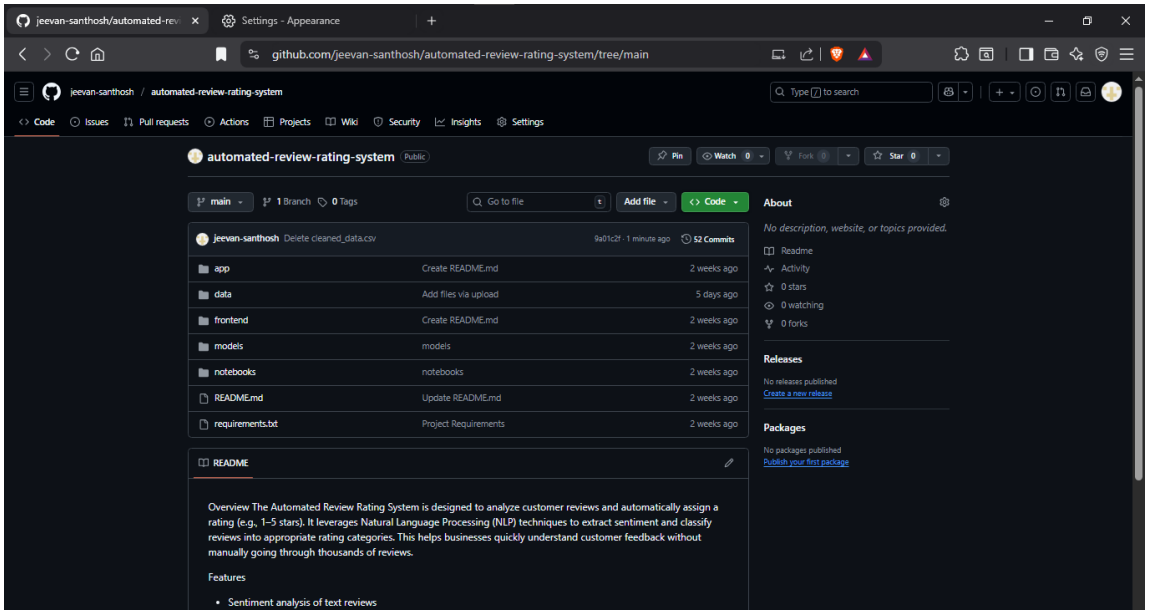
2. Environment Setup

- Installed Python 3.10+ with required libraries: pandas, numpy, matplotlib, seaborn, spacy, scikit-learn, beautifulsoup4, requests.
- IDE used: VS Code
- Verified proper functioning of packages for data preprocessing, NLP, and visualization.

3. GitHub Project Setup

Created a GitHub repository **automated-review-rating-system**

Structure of the directory



4. Data Collection

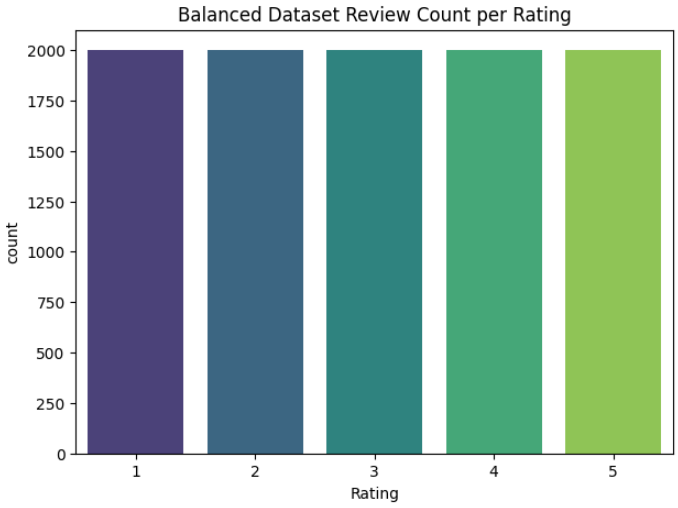
- Data was collected through from Kaggle

- Data collected from Kaggle had 192178 rows and it was dataset related to flipkart reviews

- Dataset link – Expanded Dataset

4.1 Balanced Dataset

- Link for balanced dataset : Balanced Dataset
- Balanced Dataset was created with 2000 row of each rating.



5. Data Preprocessing

Effective data preprocessing is essential to improve the performance and accuracy of machine learning models. The following techniques were applied to prepare the raw review dataset for modeling.

5.1 Removing Duplicates

Duplicate rows containing the exact same review and rating were removed to prevent bias and overfitting. This ensured that the dataset only included unique observations.

Code :

```
expanded_dataset.drop_duplicates(subset=["Review"], inplace=True)
```

5.2 Removing Conflicting Reviews

Some reviews had identical text but different star ratings. These inconsistencies can confuse the model. Such conflicting entries were identified and removed to maintain label clarity.

Code :

```
conflict_reviews =
expanded_dataset.groupby("Review")["Rating"].nunique()
conflict_reviews = conflict_reviews[conflict_reviews > 1].index

expanded_dataset =
expanded_dataset[~expanded_dataset["Review"].isin(conflict_reviews)]
```

5.3 Handling Missing Values

Rows with missing or null values-particularly in the review text or rating column-were removed. This step ensured the dataset was complete and meaningful for analysis.

Code:

```
expanded_dataset.dropna(subset=["Review", "Rating"], inplace=True)
```

5.4 Dropping Unnecessary Columns

Non-essential columns such as review IDs, timestamps, or user identifiers were dropped. These fields did not contribute to the model and could introduce noise or privacy concerns. Rows with missing or null values particularly in the review text or rating column were removed. This step ensured the dataset was complete and meaningful for analysis

Code:

```
expanded_dataset = expanded_dataset[["ID", "Rating", "Review"]].copy()
```

5.5 Lowercasing Text

All review text was converted to lowercase to maintain uniformity. This helps prevent duplication of tokens like "Good" and "good" being treated as separate words.

Code :

```
text = str(text).lower()
```

5.6 Removing URLs

URLs present in the review text were removed using regular expressions.

Code :

```
text = re.sub(r'http\S+|www.\S+', '', text)
```

5.7 Removing Emojis and Special Characters

Emojis and special symbols may not contribute meaningful context for text analysis (unless specifically required). We remove these characters to focus on the core textual content.

Code :

```
emoji_pattern = re.compile("["
                                u"\U0001F600-\U0001F64F"
                                u"\U0001F300-\U0001F5FF"
                                u"\U0001F680-\U0001F6FF"
                                u"\U0001F1E0-\U0001F1FF"
                                "]+", flags=re.UNICODE)
    text = emoji_pattern.sub(r'', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

5.8 Removing Puncuations

Punctuation marks and numbers can disrupt the natural language patterns of the text. Hy removing then, we simplify the tokenization process and prevent punctuation from being misinterpreted as separate tokens.
Code :

```
text = re.sub(r'<.*?>', '', text)
```

5.9 Stopwords Removal

Stopwords, such as "the", "is", and "and", are frequently occurring words that might not add significant semantic value. We use libraries like Spacy to filter these words out, reducing noise and focusing on words that contribute meaning to the reviews, there were total 224 stop words in spacy.

```
("like 'a', 'about', 'above', 'after', 'again', 'against', 'ain',
'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't",'as', 'at',
'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both',
'but', 'by', 'can', 'couldn', "couldn't", 'd', 'did', 'didn',"didn't",
'do', 'does', 'doesn', "doesn't", 'doing', 'don', "don't", 'down',
'during', 'each', 'few', 'for', 'from', 'further', 'had',
'hadn',”hadn't", 'has', 'hasn', "hasn't", 'have', 'haven', "haven't",
'having', 'he', "he'd", "he'll", "he's", 'her', 'here', 'hers',
'herself', 'him', 'himself', 'his', 'how', 'i', "i'd", "i'll", "i'm",
"i've", 'if', 'in', 'into', 'is', 'isn', "isn't", 'it', "it'd",
```

```
"it'll", "it's", 'its', 'itself', 'just', 'll', 'm', 'ma', 'me',
'mightn', "mightn't", 'more', 'most', 'mustn', "mustn't", 'my',
'myself', 'needn', "needn't", 'no', 'nor', 'not', 'now', 'o', 'of',
'off', 'on','once', 'only', 'or', 'other', 'our', 'ours', 'ourselves',
'out', 'over', 'own', 're', 's', 'same', 'shan', "shan't", 'she',
"she'd", "she'll", "she's",'should', "should've", 'shouldn',
"shouldn't", 'so', 'some', 'such', 't', 'than', 'that', "that'll",
'the', 'their','theirs', 'them', 'themselves','then', 'there', 'these',
'they', "they'd", "they'll", "they're", "they've", 'this', 'those',
'through', 'to', 'too', 'under', 'until', 'up', 've', 'very','was',
'wasn’t', "wasn't", 'we', "we'd", "we'll", "we're", "we've", 'were',
'weren', "weren't", 'what', 'when', 'where', 'which', 'while','who',
'whom', 'why', 'will', 'with', 'won', "won't", 'wouldn', "wouldn't",
'y', 'you', "you'd", "you'll", "you're", "you've", 'your', 'yours',
'yourself', 'yourselves')"
```

Code:

```
import nltk
nltk.download('stopwords')
nltk.download('punkt')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import spacy

nlp = spacy.load('en_core_web_sm')
nltk_stopwords = set(stopwords.words('english'))

def preprocess_text(text):
    doc = nlp(text)
    tokens = [token.lemma_ for token in doc if token.text not in
nltk_stopwords and token.is_alpha]
    return " ".join(tokens)
```

5.10 Lemmatization

Lemmatization is a text preprocessing technique that reduces words to their base or dictionary form, known as the lemma. For example, words like "running", "ran", and "runs" are all reduced to the base form "run". Unlike stemming, lemmatization uses linguistic rules and vocabulary, ensuring that the resulting word is meaningful. This helps in grouping similar words and improves model performance by reducing the size of the vocabulary without losing context.

Why lemmatization is better than stemming?

Lemmatization is often preferred over stemming because:

1. Produces Real Words:
Lemmatization returns valid dictionary words (e.g., "running" "run"), whereas stemming may return incomplete or non-existent words (e.g., "running" → "runn").

2. **Context-Aware:**
   Lemmatization considers the context and part of speech (POS) of a word to find its correct root form.
   Stemming blindly trims word endings without understanding grammar.

3. **More Accurate and Clean:**
   Lemmatization provides cleaner and more accurate tokens, which improves model understanding and performance, especially in NLP tasks like sentiment analysis or classification.

Code :

```
def preprocess_text(text):
    doc = nlp(text)
    tokens = [token.lemma_ for token in doc if token.text not in
nltk_stopwords and token.is_alpha]
    return " ".join(tokens)
```

5.11 Filtering by Word Count

Very short reviews might not contain enough context to be useful, and excessively long reviews could be outliers. We apply filtering to exclude:

- Reviews with fewer than 3 words.
- Reviews that exceed 100 words.
  This ensures that the dataset remains robust and relevant for model training.

Code :

```
before_word_filter = len(expanded_dataset)

expanded_dataset = expanded_dataset[expanded_dataset["Review"].apply
                          (lambda x: 3 <= len(str(x).split()) <=
                          10)]

after_word_filter = len(expanded_dataset)
```

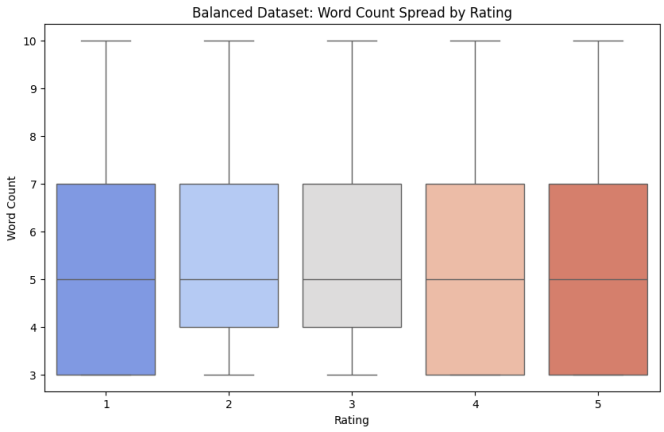
6. Data Visualization

Bar Plot

A bar plot is a chart that represents categorical data with rectangular bars, where the height of each bar indicates the value or frequency of that category. In this project, bar plots were used to show the number of reviews per rating class (1 to 5 stars), helping to visualize class distribution and detect any imbalance in the dataset.

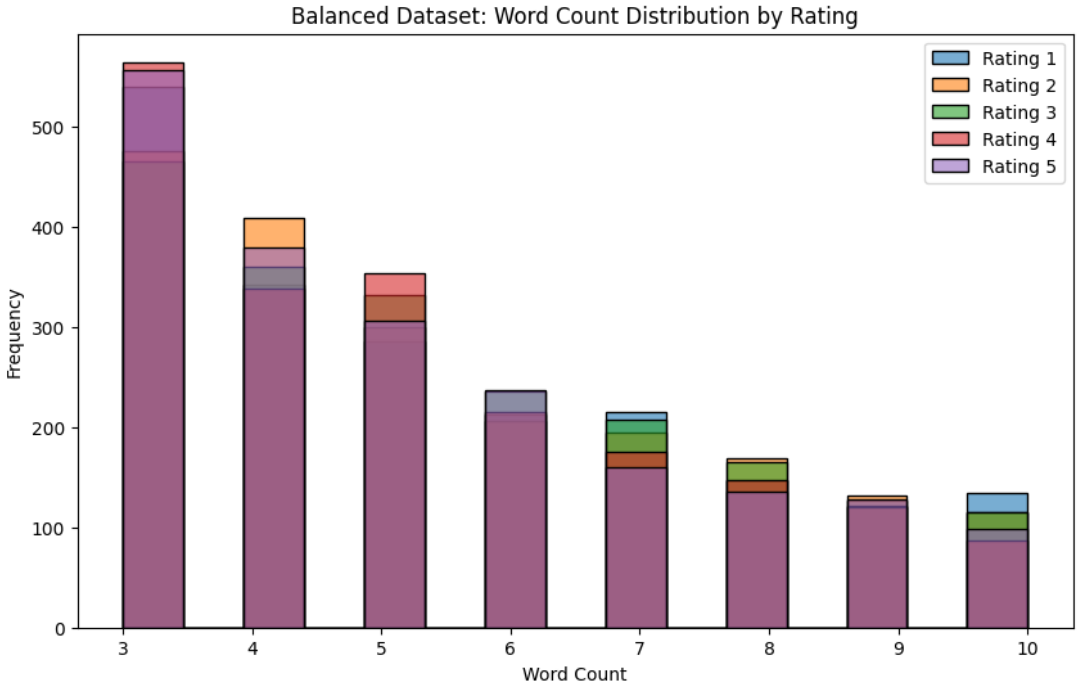
Box Plot

A box plot is a visual tool used to display the distribution of numerical data, showing the median, quartiles, and potential outliers. It helps quickly understand how data is spread and identify any extreme values. In this project, box plots were used to compare the word count distribution of reviews across different rating classes.



Histogram

A histogram in a graphical representation that shows the distribution of a numeric variable by dividing the data into intervals (bins) and counting how many values fall into each bin, in this project, histograms were used to visualize the word count distribution of reviews, helping to identify common review lengths and spot patterns across different rating levels.



Samples of 1 star Rating

Showing 5 sample reviews for Rating 1 :

1. very cheap quality after washing it faded

2. not up to the mark
3. tree very poor quality very small size

4. cant connect wifi

5. very bad waste of money not working

Samples of 2 stars Rating

Showing 5 sample reviews for Rating 2 :

1. 2 hrs more than heating storage

2. no best quality in bedsheet

3. very bad

4. very low material use

5. battery backup is not so good

Samples of 3 stars Rating

Showing 5 sample reviews for Rating 3 :

1. little bit big in sizes looks odd

2. nice product but cost is more for this product quantity

3. good value for money

4. range of router is very weak

5. size of plates and bowls are comparatively small

Samples of 4 stars Rating

Showing 5 sample reviews for Rating 4 :

1. good for 4 to 7 years kids

2. value for money

3. value for money

4. super cute and amazing

5. v good but v small

Samples of 5 stars Rating

Showing 5 sample reviews for Rating 5 :

1. very nice job

2. reserve for new home

3. best this price

4. recommended to this product good

5. bass sound better quality design very attractive

7. Train Test Split

Train-Test Split is a technique to divide the dataset into two separate sets:

- Training Set (typically 80%): Used to train the machine learning model.
- Test Set (typically 20%): Used to evaluate the model's performance on unseen data.

This separation ensures that the model generalizes well and is not just memorizing the training data.

Why Stratified Split?

In classification problems like review rating prediction, stratified splitting is important to maintain class balance (equal distribution of ratings) in both training and testing sets.

Without stratification, some rating classes (like 1-star or 5-star) may be underrepresented in the test set, leading to biased evaluation.

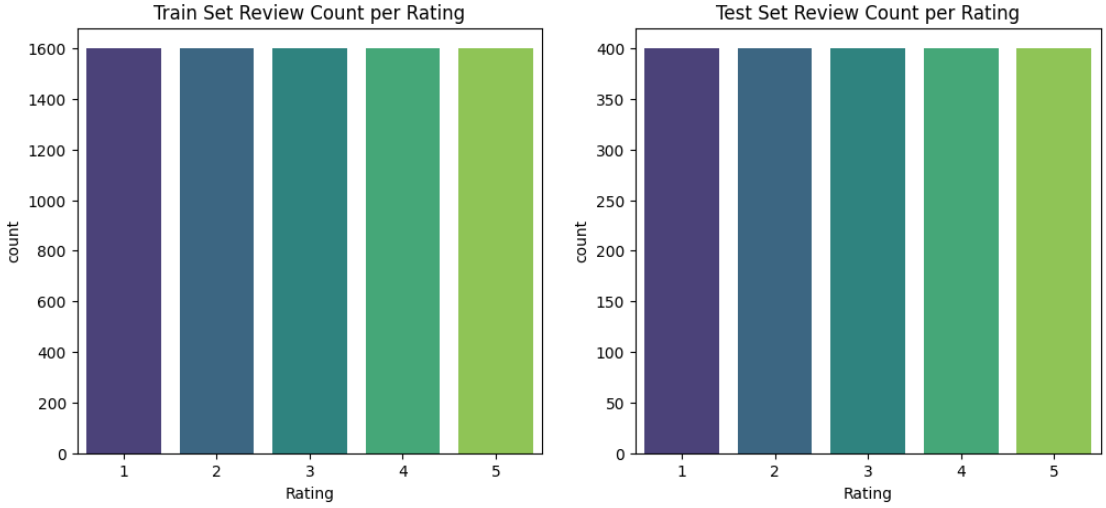
Code :

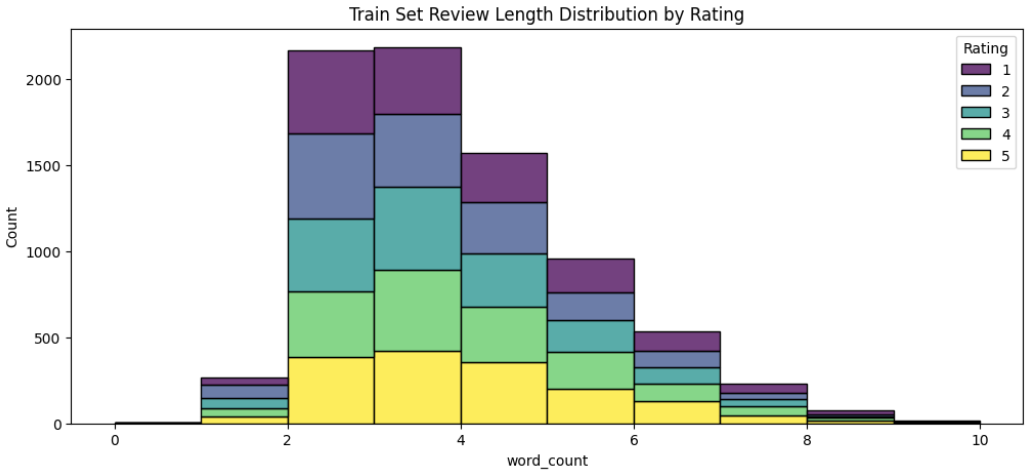
```
# Stratified split (80% train, 20% test)
X = balanced_dataset['Review']
y = balanced_dataset['Rating']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

print(f"Training set shape: {X_train.shape}")
print(f"Test set shape: {X_test.shape}")
```

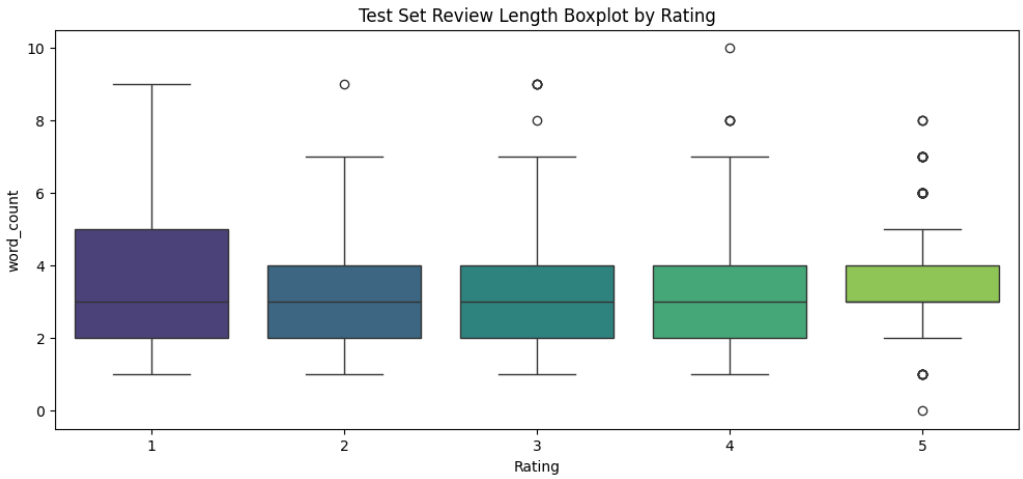
How It Was Done:

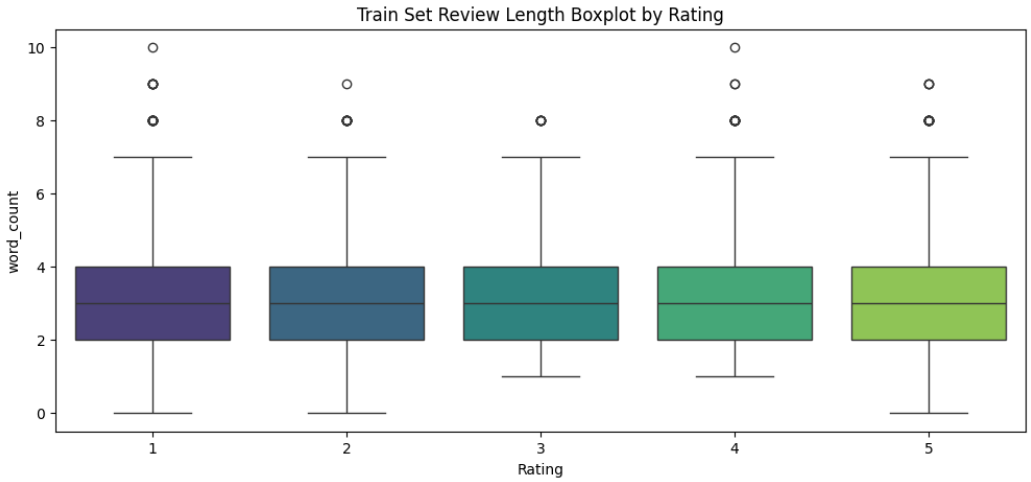
To prepare the data for model training, the dataset was first shuffled randomly to eliminate any order bias. A stratified train-test split was then performed using train teat split from rkimarn.model selection with the ratify-y argument to ensure that all star ratings were proportionally represented in both training and testing sets. The dataset was split using an 80% training and 20% testing ratio. After splitting, all text preprocessing stops-Including lowercasing. Iemmatization, stopword removal, and

cleaning were applied separately tox train and x text to prevent data leakage and maintain model integrity.









8. Text Vectorization

Machine learning models can't work directly with raw text-they require numerical input. Vectorization is the process of converting text data into numerical features.

TF-IDF (Term Frequency - Inverse Document Frequency)

TF-IDF is a statistical technique that represents how important a word is to a document relative to the entire corpus.

- TF (Term Frequency): How often a word appears in a document.
- IDF (Inverse Document Frequency): Penalizes common words and highlights rare but Important ones.

This approach helps to capture both word relevance and discriminative power, making it better than simple word counts.

Formula for TF-IDF:

$$TF(t,d) = \frac{number\ of\ times\ t\ appears\ in\ d}{total\ number\ of\ terms\ in\ d}$$

$$IDF(t) = log\frac{N}{1+df}$$

$$TF - IDF(t,d) = TF(t,d) * IDF(t)$$

Code :

```
tfidf = TfidfVectorizer(max_features=5000)
```
